

Solving the Stochastic Liouville Equation

Deval Deliwala

July 9, 2025

In our previous notes, we derived the spin Hamiltonian $\mathcal{H}$ for 4H-Silicon Carbide, which captures the principal spin interactions when 4H-SiC is subjected to an external magnetic field. We then computed eigenspectra for several model Hamiltonians, including the complete 16×16 representation of $\mathcal{H}$.

Now, our goal is to build a first-principles model of Electrically-Detected Magnetic Resonance (EDMR) and Near-Zero Field Magnetic Resonance (NZFMR). Specifically, we want to reproduce the experimental trace shown in Figure 1 using simulation.

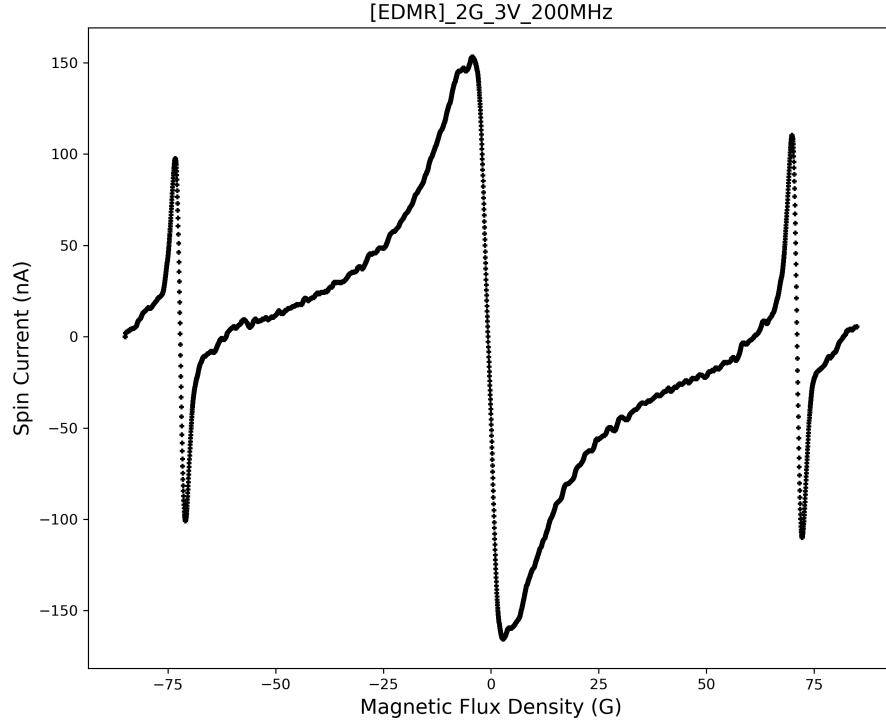


Figure 1: I (nA) vs. B (G) EDMR spectra for 4H-SiC at 2 G modulation amplitude, 200 MHz, and 3 V bias.

To achieve this, the model must accept both the intrinsic Hamiltonian parameters (Zeeman, hyperfine, zero-field splitting, exchange) and the experimental conditions (bias voltage, modulation amplitude, etc.) as inputs. Next, we specify the magnetic field range via a tuple $(B_{\min}, B_{\max})$ and perform the sweep to reproduce the reference spectrum precisely. The framework should remain flexible, allowing substitution of other experimental parameters. This will be a long battle, so I will split up this note into three documents:

1. Solving the Stochastic Liouville Equation (SLE).
2. Building the EDMR model.
3. Least Squares + MCMC fitting the EDMR model.

Contents

1	The Stochastic Liouville Equation	3
1.1	Density Matrix Formalism	3
1.2	Quantum Liouville Equation	3
2	Solving the Stochastic Liouville Equation	5
2.1	Sylvester Form	5
3	Modeling Sylvester	6
3.1	Vectorization	7
3.1.1	Checks	8
4	Connecting back to EDMR/NZFMR	9

1 The Stochastic Liouville Equation

1.1 Density Matrix Formalism

The time evolution of a wave function $|\psi\rangle$ is calculated by solving the time-dependent Schrödinger equation:

$$|\Psi(t)\rangle = e^{(-\frac{i}{\hbar}\hat{H}t)}|\psi\rangle. \quad (1)$$

The *density operator* is defined as

$$\hat{\rho}(t) = \sum_i W_i |\Psi_i(t)\rangle \langle \Psi_i(t)| \quad (2)$$

where W_i represents the statistical weight of being in the state $|\Psi_i\rangle$ within the system. In an orthonormal basis $\{\phi_i\}$, the *density matrix* is given by

$$\rho_{i,j} = \langle \phi_i | \hat{\rho} | \phi_j \rangle. \quad (3)$$

1.2 Quantum Liouville Equation

The *Quantum Liouville Equation* (QLE) is just the Schrödinger equation but in density matrix representation.

Substituting Equation 1 into Equation 2, we get

$$\hat{\rho}(t) = \sum_i W_i e^{(-\frac{i}{\hbar}\hat{H}t)} |\psi_i\rangle \langle \psi_i| e^{(\frac{i}{\hbar}\hat{H}t)} \quad (4)$$

$$= \sum_i W_i \hat{U}(t) |\psi_i\rangle \langle \psi_i| \hat{U}^\dagger(t) \quad (5)$$

Considering an orthonormal basis, we get the time-dependent density matrix from Equation 3:

$$\rho(t) = \hat{U}(t) \rho \hat{U}^\dagger(t) \quad (6)$$

$$= e^{(-\frac{i}{\hbar}\hat{H}t)} \rho e^{(\frac{i}{\hbar}\hat{H}t)} \quad (7)$$

The QLE, describes the time evolution of the density ensemble can be found by taking the derivative of Equation 6 w.r.t time:

$$i\hbar \frac{\partial}{\partial t} \rho(t) = i\hbar \frac{\partial \hat{\mathcal{U}}(t)}{\partial t} \rho \hat{\mathcal{U}}^\dagger(t) + i\hbar \hat{\mathcal{U}}(t) \rho \frac{\partial \hat{\mathcal{U}}^\dagger(t)}{\partial t} \quad (8)$$

$$i\hbar \frac{\partial}{\partial t} \rho(t) = i\hbar \frac{\partial e^{(-\frac{i}{\hbar} \hat{\mathcal{H}} t)}}{\partial t} \rho \hat{\mathcal{U}}^\dagger(t) + i\hbar \hat{\mathcal{U}}(t) \rho \frac{\partial e^{(\frac{i}{\hbar} \hat{\mathcal{H}} t)}}{\partial t} \quad (9)$$

$$i\hbar \frac{\partial}{\partial t} \rho(t) = \hat{\mathcal{H}} \left(\hat{\mathcal{U}}(t) \rho \hat{\mathcal{U}}^\dagger(t) \right) + \left(\hat{\mathcal{U}}(t) \rho \hat{\mathcal{U}}^\dagger(t) \right) \hat{\mathcal{H}} \quad (10)$$

$$i\hbar \frac{\partial}{\partial t} \rho(t) = \hat{\mathcal{H}}(\rho(t)) + (\rho(t)) \hat{\mathcal{H}} \quad (11)$$

$$i\hbar \frac{\partial}{\partial t} \rho(t) = [\hat{\mathcal{H}}, \rho(t)]. \quad (12)$$

This equation describes the unitary, non-dissipative evolution of the spin ensemble. To include environmental interactions, we must generalize to the Stochastic Liouville Equation (SLE):

$$\frac{\partial \rho}{\partial t} = -\frac{i}{\hbar} [\mathcal{H}, \rho] - \frac{k}{2} \{\Lambda, \rho\} + p\Gamma. \quad (13)$$

The first term on the right-hand side reproduces the unitary QLE. The second term introduces dissipation: at rate k , the projection operator Λ removes spin pairs from the corresponding subspace. The third term acts as a source, injecting spin pairs with random orientations at rate p . Here, Γ denotes the 16×16 identity matrix. This formulation provides the basis for computing the various spin-spin reactions that give rise to the observed EDMR and NZFMR spectra.

By substituting our full spin Hamiltonian into Equation 13 and applying a small modification, we capture spin-dependent recombination (SDR) and thus derive a model for EDMR and NZFMR simulations.

We adopt the modified SLE of Hansen and Pedersen¹:

$$\frac{\partial \rho}{\partial t} = -\frac{i}{\hbar} [\mathcal{H}, \rho] - \frac{1}{2} (k_S + k_D) \{\Lambda_S, \rho\} - \frac{1}{2} k_D \{\Lambda_T, \rho\} + \frac{1}{16} p \Gamma. \quad (14)$$

Again, the first term is the unitary evolution. The second term, via Λ_S , removes singlet states at the effective rate $(k_S + k_D)$. The third term, via Λ_T , accounts for triplet dissociation into the conduction band at rate k_D . The final term injects carriers at rate p , with Γ the identity; the factor $1/16$ normalizes this over the 16-dimensional space.

Since carrier generation and annihilation proceed at constant rates, we focus on the steady-state SLE ($\partial \rho / \partial t = 0$). Hence, we solve

$$0 = -\frac{i}{\hbar} [\mathcal{H}, \rho] - \frac{1}{2} (k_S + k_D) \{\Lambda_S, \rho\} - \frac{1}{2} k_D \{\Lambda_T, \rho\} + \frac{1}{16} p \Gamma. \quad (15)$$

¹<https://www.sciencedirect.com/science/article/abs/pii/S0009261402007248?via%3Dihub>

2 Solving the Stochastic Liouville Equation

2.1 Sylvester Form

We consider a pair of electrons (spin- $\frac{1}{2}$) coupled to two spin- $\frac{1}{2}$ nuclei ($^{29}\text{Si} + ^{13}\text{C}$). The composite Hilbert space is

$$\mathcal{H} = \left(\frac{1}{2}\text{-electron}\right)^{\otimes 2} \otimes \left(\frac{1}{2}\text{-nucleus}\right)^{\otimes 2}, \quad \dim \mathcal{H} = 2^4 = 16.$$

We have a symbolically parameterized Spin Hamiltonian (from the previous notes):

$$\hat{\mathcal{H}}_{\text{sym}} \in \mathbb{C}^{16 \times 16}$$

that contains Zeeman, Hyperfine, Zero-field Splitting, and Exchange terms.

The SDR model augments the unitary QLE-von Neumann equation with three dissipative channels

$$0 = -\frac{i}{\hbar}[\mathcal{H}, \rho] - \frac{1}{2}(k_S + k_D)\{\Lambda_S, \rho\} - \frac{1}{2}k_D\{\Lambda_T, \rho\} + \frac{1}{16}p\Gamma, \quad (16)$$

where

- . $[\mathcal{H}, \rho]$ - coherent precession.
- . $\{\Lambda_S, \rho\}$ - recombination of *singlet* electron pairs into the valence band.
- . $\{\Lambda_T, \rho\}$ - dissociation of *triplet* electron pairs into the conduction band.
- . $\frac{1}{16}p\Gamma$ - spin-pair generation at rate p uniformly over the 16-dimensional space.

Equation 16 is a *linear, time-homogeneous, trace non-conserving* master equation that can be rearranged into a **Sylvester Equation**.

We rewrite Equation 16 into Sylvester form (more specifically, a continuous Lyapunov) via

$$A\rho + \rho A^\dagger + g = 0 \quad (17)$$

where

$$A = -\frac{i}{\hbar}\mathcal{H} - \frac{1}{2}(k_S + k_D)\Lambda_S - \frac{1}{2}k_D\Lambda_T, \quad g = -\frac{1}{16}p\Gamma. \quad (18)$$

The steady state is a unique solution **if and only if** A has no purely imaginary eigenvalues.

3 Modeling Sylvester

The solution of Equation 17 is given by

$$\rho = \int_0^\infty e^{-iAt} g e^{iA^\dagger t} dt.$$

Although this integral form is formally exact, numerical implementations rely on matrix operations. We therefore vectorize Equation 17 into a linear system.

We begin by defining a solver class for the steady-state SLE. The class accepts the symbolic spin Hamiltonian, the singlet/triplet projectors, and the 16×16 identity matrix Γ .

```

1 @dataclass
2 class SteadyStateSLESolver:
3     """
4     Build the singlet-triplet Liouvillian for a 2-electron + 2-nucleus (I=1/2)
5     spin system numerically. Returns a callable
6
7     rho_numeric(**params) -> 16x16 numpy array (trace=1)
8
9     We use a modified spin-dependent recombination (SDR) model from Hansen
10    and Pedersen. The steady-state equation is
11
12    0 = -(i/$\hbar$)[H,$\rho$]
13        - 1/2($k_S + k_D$){$\Lambda_S$, $\rho$}
14        - 1/2$k_D${$\Lambda_T$, $\rho$}
15        + (p/16)$\Gamma$.
16
17    Here, $\Lambda_S$ and $\Lambda_T$ project onto the electronic singlet
18    and triplet subspaces (with identity on the nuclear sector). $\Gamma$
19    is the 16x16 identity.
20    """
21
22    H_sym:      sp.Matrix
23    Lambda_S:   sp.Matrix | ndarray
24    Lambda_T:   sp.Matrix | ndarray
25    Gamma:      sp.Matrix | ndarray
26    ...

```

To convert symbolic `sp.Matrix` objects into numerical `np.ndarray` matrices, we define a helper that applies a substitution map `subs` from symbols to float values.

```

1 def _to_numeric(M: sp.Matrix | ndarray, subs: Mapping[sp.Symbol, float]) -> ndarray:
2     """
3     Convert a sympy.Matrix into a numpy.ndarray by substituting parameters
4     and evaluating numerically.
5     """
6     if isinstance(M, sp.Matrix):
7         return np.array(M.subs(subs).evalf(), dtype=complex)
8     return np.array(M, dtype=complex)

```

With this in place, we assemble the Sylvester operator A and the generation term g .

```

1 # build numerical A operator in Sylvester eq.
2 def _build_A(self, *, k_s: float, k_d: float, subs: Mapping[sp.Symbol, float])
   -> ndarray:

```

```

3     H = np.array(self.H_sym.subs(subs).evalf(), dtype=complex)
4     LS = np.array(_to_numeric(self.Lambda_S, subs), dtype=complex)
5     LT = np.array(_to_numeric(self.Lambda_T, subs), dtype=complex)
6     return (
7         -(1j / self.hbar) * H
8         - 0.5 * (k_s + k_d) * LS
9         - 0.5 * k_d * LT
10    )

1    # generation term
2    def _build_g(self, *, p: float, subs: Mapping[sp.Symbol, float]) -> ndarray:
3        Gamma = np.array(_to_numeric(self.Gamma, subs), dtype=complex)
4        return (p / 16.0) * Gamma

```

3.1 Vectorization

To turn Equation 17 into a *linear* problem we vectorize the density matrix. Let $\text{vec}(\rho)$ be the 256-component column vector obtained by stacking the *columns* of ρ . Using the identity

$$\text{vec}(X\rho Y) = (Y^T \otimes X) \text{vec}(\rho),$$

Equation 17 becomes

$$\underbrace{(I \otimes A + A^\dagger \otimes I)}_{M \in \mathbb{C}^{256 \times 256}} \text{vec}(\rho) = -\text{vec}(g). \quad (19)$$

Equation 19 is a standard linear system that can be solved with any dense solver. I use `np.linalg.solve` with fortran column ordering.

```

1    def rho(
2        self, *,
3        k_s: float,
4        k_d: float,
5        p: float,
6        params: Mapping[str, float] | None = None,
7    ) -> ndarray:
8        """
9        Return the steady-state density matrix (16x16, complex Hermitian).
10
11        * Hamiltonian parameters are supplied through 'params'.
12        * Kinetic parameters are 'k_s', 'k_d', and 'p'.
13
14        Args
15        ----
16        k_s, k_d : float
17            Singlet recombination and triplet dissociation rates (s^-1).
18        p : float
19            Spin-pair generation rate (s^-1).
20        params : dict[str, float], optional
21            Numeric values for every free symbol in 'H_sym'
22            (B0, g_e, mu_B, ...).
23        """
24        if params is None:
25            params = {}
26        subs = {sp.Symbol(k): v for k, v in params.items()}

```

```

27     self._check_param_completeness(subs)
28
29     A = self._build_A(k_s=k_s, k_d=k_d, subs=subs)    # (16, 16)
30     self._spectral_check(A)
31
32     B = A.conj().T                                     # (16, 16)
33
34     g = self._build_g(p=p, subs=subs)                 # (16, 16)
35     I = np.eye(self.N, dtype=complex)                 # (16, 16)
36     M = kron(I, A) + kron(B.T, I)                     # (256, 256)
37     self._structure_checks(M, A, B)
38
39     # solve for vec(rho)
40     vec_rho = np.linalg.solve(M, -g.reshape(-1, 1, order="F"))
41
42     # reshape 256x1 -> 16x16
43     rho = vec_rho.reshape(self.N, self.N, order="F")
44     rho = self._postprocess(rho)
45     return rho

```

For any combination of Hamiltonian and experimental parameters (including a specific magnetic-field value B), calling `SteadyStateSLESolver.rho(...)` returns the steady-state density matrix of the full spin ensemble. To guarantee numerical correctness the solver performs several checks.

3.1.1 Checks

1. Dimension check. Ensure every operator supplied to the solver has the correct 16×16 shape:

```

1     def _check_dimensions(self):
2         for name, M in (
3             ("H", self.H_sym),
4             ("Lambda_S", self.Lambda_S),
5             ("Lambda_T", self.Lambda_T),
6             ("Gamma", self.Gamma),
7         ):
8             if M.shape != (self.N, self.N):
9                 raise ValueError(f"{name} must be {self.N}x{self.N}; got {M.shape}.")
10

```

2. Parameter completeness. Confirm the substitution dictionary covers every free symbol:

```

1     def _check_param_completeness(self, subs: Mapping[sp.Symbol, float]):
2         missing = set(self._free_params) - set(subs)
3         if missing:
4             raise ValueError(
5                 "numeric values missing for: " + ", ".join(s.name for s in missing)
6             )

```

3. Spectral check. The Sylvester equation is solvable only if A has no eigenvalues with real part 0. A numerical margin of 10^{-10} is enforced:

```

1     def _spectral_check(self, A: ndarray):
2         lam = eigvals(A)
3         if np.any(np.abs(np.real(lam)) < 1e-10):
4             raise ValueError(

```



```

5         "steady state solution must be unique; some eigenvalues are near 0."
6     )
7     return lam

```

4. Structure check. After vectorization we verify the Kronecker-sum matrix, its rank, and the relation $B = A^\dagger$:

```

1     def _structure_checks(self, M: ndarray, A: ndarray, B: ndarray):
2         N2 = self.N * self.N # 256
3         if M.shape != (N2, N2):
4             raise RuntimeError("Kronecker sum matrix M has wrong dimensions.")
5         if np.linalg.matrix_rank(M) < N2:
6             raise RuntimeError("M is singular; steady state not unique or absent.")
7         if not np.allclose(B, A.conj().T):
8             raise RuntimeError("B != A^dagger")

```

5. Post-processing. After reshaping the solution back to 16×16 we enforce Hermiticity, normalise the trace, and check positivity:

```

1     def _postprocess(self, rho: ndarray) -> ndarray:
2         # enforce Hermiticity
3         rho = 0.5 * (rho + rho.conj().T)
4
5         # trace normalisation
6         tr = np.trace(rho)
7         if not np.isclose(tr, 1.0, atol=1e-8):
8             rho /= tr
9
10        # positivity (tiny negatives tolerated)
11        evals = eigvalsh(rho)
12        if np.any(evals < -1e-8):
13            raise RuntimeError("rho not positive semi-definite (min eigenvalue < 0).")
14
15    return rho

```

The first line explicitly projects out any anti-Hermitian numerical noise so that ρ is *exactly* Hermitian before further tests.

The resulting $\rho(B)$ is the steady-state solution of Equation 13. Sweeping B from $(B_{\min}, B_{\max})$ yields the full density matrix—and therefore a complete description of the spin ensemble—at each field point.

4 Connecting back to EDMR/NZFMR

The measured EDMR/NZFMR signal is proportional to the singlet population in the sample, since only singlets ($S = 0$) can recombine by spin-selection rules (spin angular momentum conservation). Hence, we compute

$$P_S(B) = \text{Tr}(\Lambda_S \rho(B)),$$

where Λ_S is the singlet projector on the 16×16 Hilbert space. Finally the EDMR and NZFMR spin current can be modeled via

$$I(B) \propto P_S(B) \implies I(B) = A \text{Tr}(\Lambda_S \rho(B)) + I_0,$$

where A is a scaling constant and I_0 a small offset. Determining A and I_0 by fitting will be the subject of a future note.